



Praxisorientierte Fallstudie:

Nutzwertanalyse von End-to-End-Test-Frameworks (Selenium vs. Cypress) für die Automatisierung repräsentativer regressionskritischer Testszenarien zur Reduzierung des manuellen Testaufwands

Name: Marcel Licht

Studiengang: Bachelor Angewandte Informatik dual

Modul: Aktuelles Thema der Angewandten Informatik – Praxisorientierte Fallstudie

Matrikelnummer: 1540372

Lehrperson: Prof. Dr. Martine Herpers

Unternehmen: JUMO GmbH & Co. KG

Fachbetreuung: Daniel Kircher

Datum: 4. Juli 2026

Zusammenfassung

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.

Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero eros et accumsan et iusto odio dignissim qui blandit praesent luptatum zzril delenit augue dui dolore te feugait nulla facilisi. Lorem ipsum dolor sit amet,

Inhaltsverzeichnis

Abbildungsverzeichnis	5
Tabellenverzeichnis	6
Quellcodeverzeichnis	7
1 Einleitung	8
1.1 Problemstellung und Motivation	8
1.2 Zielsetzung der Arbeit	8
1.3 Forschungsfrage und Aufbau der Arbeit	9
2 Theoretische Grundlagen	10
2.1 Stand der Forschung	10
2.2 Softwaretest und End-to-End-Testing	10
2.3 Betrachtete Test-Frameworks: Selenium und Cypress	10
2.4 Nutzwertanalyse als Bewertungsmethode	11
3 Methodisches Vorgehen	12
3.1 Forschungsdesign der Fallstudie	12
3.2 Auswahl repräsentativer regressionskritischer Testszenarien	12
3.3 Auswahl repräsentativer regressionskritischer Testszenarien	12
3.4 Ableitung und Gewichtung der Bewertungskriterien	13
3.5 Vorgehen der prototypischen Umsetzung	13
3.6 Datenerhebung und Auswertung	14
4 Prototypische Umsetzung und Datenerhebung	15
4.1 Das Testobjekt: Die Webanwendung „Puma“	15
4.2 Testkonzeption und Vergleichbarkeit	15
4.3 Implementierung mit Selenium	16
4.4 Implementierung mit Cypress	16
4.5 Datenerhebung und praktische Beobachtungen	17
5 Ergebnisse und Nutzwertberechnung	18
5.1 Präsentation der quantitativen Messdaten	18
5.2 Mathematische Auswertung (Nutzwertmatrix)	18
5.3 Analyse und Diskussion der Ergebnisse	18
6 Fazit und Handlungsempfehlung	19

6.1	Zusammenfassung der Erkenntnisse	19
6.2	Handlungsempfehlung für die JUMO GmbH & Co. KG	19
6.3	Ausblick und zukünftige Schritte	19
Anhänge		20
A	Anhang	20
A.1	Anhang A	20
Literaturverzeichnis		21

Abbildungsverzeichnis

Tabellenverzeichnis

Quellcodeverzeichnis

1 Einleitung

1.1 Problemstellung und Motivation

Das Testen von Software ist ein zentraler Bestandteil moderner Softwareentwicklung, um funktionale Anforderungen und einen zuverlässigen Betrieb von Anwendungen sicherzustellen. Historisch wurden Tests überwiegend manuell durchgeführt. Mit der zunehmenden Komplexität heutiger Softwaresysteme und steigenden Anforderungen an Entwicklungsgeschwindigkeit hat sich jedoch der Einsatz automatisierter Testverfahren etabliert [1].

Im betrieblichen Kontext der JUMO GmbH & Co. KG wird die intern entwickelte Java-Webanwendung „Puma“ (auch „Workspace“ genannt) zur Prozessverwaltung fortlaufend durch Fehlerbehebungen und neue Funktionen erweitert. Trotz der Relevanz automatisierter Tests erfolgen die Prüfungen zentraler Nutzungsszenarien derzeit noch überwiegend manuell – zunächst durch die Entwickler im Testsystem und anschließend als Abnahmetest durch die anfordernden Fachabteilungen. Dies geschieht häufig in Form explorativer Tests.

Dieser manuelle Testaufwand ist zeitintensiv, bindet personelle Ressourcen auf beiden Seiten und erschwert eine strukturierte Absicherung von Änderungen. Zudem schwanken die Testergebnisse stark je nach durchführender Person, was die Reproduzierbarkeit einschränkt. Vor diesem Hintergrund entsteht der Bedarf, geeignete Ansätze zur teilweisen Automatisierung regressionskritischer End-to-End-Tests (E2E-Tests) systematisch zu untersuchen – also jener Testszenarien, die nach Änderungen besonders anfällig für unerwünschte Seiteneffekte sind.

Erschwerend müssen bei der Werkzeugauswahl spezifische infrastrukturelle Rahmenbedingungen der JUMO GmbH & Co. KG berücksichtigt werden. Dazu zählen die bestehende Entwicklungslandschaft (älteres JDK, Eclipse, Maven) sowie strikte Proxy- und Netzwerksperren des Unternehmens. Im Fokus dieser Arbeit stehen daher die E2E-Frameworks Selenium [2] und Cypress [3].

Die Motivation der Arbeit ergibt sich somit aus dem konkreten betrieblichen Bedarf, die Qualitätssicherung bei häufigen Softwareänderungen effizienter und reproduzierbarer zu gestalten.

1.2 Zielsetzung der Arbeit

Ziel der vorliegenden Fallstudie ist es, auf Basis eines strukturierten Vergleichs eine fundierte Entscheidungshilfe für die Auswahl eines End-to-End-Test-Frameworks bei der JUMO GmbH & Co. KG zu erarbeiten.

Dabei ist es nicht das Ziel, ein allgemein überlegenes Framework zu bestimmen, sondern das Werkzeug zu identifizieren, welches für den konkreten Unternehmenskontext und dessen technische Restriktionen am besten geeignet ist. Hierzu werden relevante Anforderungen aus dem Betrieb abgeleitet, ausgewählte Nutzungsszenarien der „Puma“-

Webanwendung prototypisch in beiden Frameworks umgesetzt und die Ergebnisse anhand einer Nutzwertanalyse systematisch bewertet.

1.3 Forschungsfrage und Aufbau der Arbeit

Aus der formulierten Zielsetzung leitet sich die zentrale Forschungsfrage dieser Fallstudie ab:

Welches der beiden Frameworks (Selenium oder Cypress) ist für die Automatisierung ausgewählter repräsentativer regressionskritischer End-to-End-Testszenarien im gegebenen Unternehmenskontext besser geeignet, wenn Integrationsaufwand, Wartbarkeit, Stabilität, Ausführungseffizienz und Unterstützungsangebote systematisch berücksichtigt werden?

Zur präzisen Beantwortung der Hauptfrage werden folgende Teilfragen untersucht:

- Welche Anforderungen ergeben sich aus dem betrieblichen Umfeld an ein End-to-End-Test-Framework?
- Welche Nutzungsszenarien der Webanwendung eignen sich als repräsentative Testfälle für einen Framework-Vergleich?
- Wie unterscheiden sich Selenium und Cypress hinsichtlich des technischen und organisatorischen Integrationsaufwands?
- Welche Unterschiede zeigen sich in Bezug auf Verständlichkeit, Wartbarkeit und Änderbarkeit des Testcodes?
- Wie unterscheiden sich die Frameworks bezüglich Stabilität, Laufzeit und Diagnosemöglichkeiten im Fehlerfall?

Aufbau der Arbeit:

Im Anschluss an diese Einleitung werden in Kapitel 2 die theoretischen Grundlagen des End-to-End-Testings sowie der untersuchten Frameworks Selenium und Cypress erläutert. Kapitel 3 widmet sich der Methodik und beschreibt das Vorgehen der Nutzwertanalyse inklusive der Entwicklung des Kriterienmodells. In Kapitel 4 erfolgt die Beschreibung der prototypischen Umsetzung und der Datenerhebung anhand der ausgewählten Testszenarien. Kapitel 5 präsentiert die Auswertung der Ergebnisse mittels der Nutzwertanalyse. Die Arbeit schließt in Kapitel 6 mit einer Diskussion der Erkenntnisse und einem zusammenfassenden Fazit.

2 Theoretische Grundlagen

Dieses Kapitel legt das theoretische Fundament für die Fallstudie. Es beleuchtet zunächst den aktuellen Stand der Forschung, definiert die Rolle von End-to-End-Tests in der Qualitätssicherung, stellt anschließend die beiden zu vergleichenden Frameworks vor und erläutert abschließend die Methodik der Nutzwertanalyse.

2.1 Stand der Forschung

Die Evaluierung und der Vergleich von Testautomatisierungswerkzeugen sind Gegenstand zahlreicher Publikationen der angewandten Informatik. Während grundlegende Werke wie Liggesmeyer [4] oder Spillner und Linz [1] die theoretischen Konzepte des Softwaretests und der Testautomatisierung umfassend behandeln, fokussieren sich neuere Publikationen häufig auf den Architekturvergleich spezifischer Frameworks. Der Kontrast zwischen traditionellen WebDriver-basierten Ansätzen wie Selenium und modernen, browserinternen Werkzeugen wie Cypress wird in der Fachliteratur oft anhand von Metriken wie Ausführungsgeschwindigkeit, Stabilität (Flakiness) und API-Design diskutiert.

Eine Lücke besteht jedoch oft in der detaillierten Betrachtung dieser Tools innerhalb stark reglementierter Unternehmensnetzwerke und spezifischer Legacy-Infrastrukturen (wie z. B. ältere JDK-Versionen oder strikte Proxy-Vorgaben). Die vorliegende Fallstudie knüpft an den bestehenden Forschungsstand an, erweitert ihn jedoch um eine systematische Bewertung, die exakt diese praxisnahen infrastrukturellen und organisatorischen Rahmenbedingungen der JUMO GmbH & Co. KG in den Mittelpunkt stellt.

2.2 Softwaretest und End-to-End-Testing

Die Testautomatisierung ist ein fest etablierter Bestandteil moderner Softwarequalitätssicherung [4]. Während Unit- und Integrationstests isolierte Code-Abschnitte oder Schnittstellen auf technischer Ebene prüfen, testen End-to-End-Tests (E2E-Tests) das gesamte System über alle Schichten hinweg – von der Benutzeroberfläche bis zur Datenbank. Sie dienen insbesondere dazu, regressionskritische Nutzungsszenarien aus Sicht der Endanwender in einer realitätsnahen Umgebung zu überprüfen und dadurch zentrale Geschäftsprozesse abzusichern [1]. Da E2E-Tests in der Regel komplexer in der Einrichtung und laufzeitintensiver sind, ist die Wahl eines robusten, wartbaren und in die Systemlandschaft passenden Automatisierungs-Frameworks entscheidend für die langfristige Akzeptanz im Entwicklungsalltag.

2.3 Betrachtete Test-Frameworks: Selenium und Cypress

Für den praktischen Vergleich dieser Arbeit stehen die Werkzeuge Selenium und Cypress im Mittelpunkt.

Selenium ist ein langjährig etabliertes Standardwerkzeug der browserbasierten Testautomatisierung. Es steuert den Browser über das native WebDriver-Protokoll an und bietet breite Integrationsmöglichkeiten in unterschiedliche Programmiersprachen, wie

beispielsweise Java [2]. Diese hohe Flexibilität und Rückwärtskompatibilität erfordern jedoch im Gegenzug oftmals einen höheren manuellen Konfigurationsaufwand, insbesondere bei der Handhabung von asynchronen Ladezeiten und der Synchronisation des Testablaufs.

Cypress wird demgegenüber häufig als moderne, entwicklerzentrierte Alternative für Webanwendungen herangezogen. Im Gegensatz zu Selenium läuft Cypress direkt im Browser und führt Tests in derselben Event-Loop wie die Anwendung selbst aus [3]. Dies ermöglicht eine direkte Rückmeldung während der Testausführung, ein automatisches Warten auf DOM-Elemente (*Auto-Waiting*) sowie sehr gute Debugging-Möglichkeiten. Da Cypress jedoch primär auf JavaScript beziehungsweise TypeScript ausgelegt ist, bedarf es bei der Integration in bestehende Java-Ökosysteme oder stark reglementierte Firmennetzwerke einer gesonderten Betrachtung.

Der Mehrwert der vorliegenden Arbeit liegt demnach nicht in einer rein allgemeinen Toolbeschreibung, sondern in der kontextbezogenen Bewertung dieser Frameworks für konkrete Webanwendungen unter realen infrastrukturellen Restriktionen im Unternehmensumfeld.

2.4 Nutzwertanalyse als Bewertungsmethode

Zur strukturierten und objektiven Entscheidungsfindung wird die Nutzwertanalyse verwendet. Diese Methode erlaubt es, mehrere Handlungsalternativen anhand systematisch gewichteter qualitativer und quantitativer Kriterien nachvollziehbar zu vergleichen [5]. Das Verfahren gliedert sich in vier zentrale Schritte: die Definition der relevanten Bewertungskriterien, deren Gewichtung auf Basis der betrieblichen Anforderungen, die Ermittlung der Zielerfüllung (Punktevergabe) durch prototypische Messungen sowie die finale Berechnung des Gesamtnutzwertes. Damit wird ein bestehender praktischer Bedarf mit einer methodisch fundierten Bewertungslogik verbunden, um eine kontextbezogene Entscheidungshilfe abzuleiten.

3 Methodisches Vorgehen

Um die Eignung der Frameworks Selenium und Cypress für den Einsatz bei der JUMO GmbH & Co. KG systematisch und objektiv zu vergleichen, wurde ein praxisorientiertes Forschungsdesign gewählt. Dieses Kapitel beschreibt den methodischen Aufbau der Fallstudie, die Auswahl der Testszenarien, die Definition der Bewertungskriterien sowie den Ablauf der praktischen Erprobung.

3.1 Forschungsdesign der Fallstudie

Die Untersuchung basiert auf einer Kombination aus prototypischer Implementierung und einer strukturierten Entscheidungsfindung mittels Nutzwertanalyse (NWA) nach Kühnapfel [5]. Da ein rein theoretischer Vergleich der Frameworks den spezifischen infrastrukturellen Kontext des Unternehmens nicht ausreichend berücksichtigen würde, werden beide Werkzeuge in der realen Entwicklungsumgebung erprobt.

Die Ergebnisse dieser praktischen Umsetzung fließen in die Nutzwertanalyse ein. Diese Methode erlaubt es, qualitative und quantitative Beobachtungen in messbare Punktwerte zu übersetzen, diese mit unternehmensspezifischen Gewichtungen zu versehen und so zu einem nachvollziehbaren Gesamtergebnis zu gelangen.

3.2 Auswahl repräsentativer regressionskritischer Testszenarien

Da eine vollständige Automatisierung aller Anwendungsfälle der „Puma“-Webanwendung den Rahmen dieser Studie sprengen würde, wurden repräsentative und regressionskritische Nutzungsszenarien ausgewählt. Diese Szenarien bilden die Kernfunktionalitäten der Anwendung ab und sind nach funktionalen Erweiterungen besonders fehleranfällig.

Für die prototypische Umsetzung wurden folgende drei Szenarien definiert:

3.3 Auswahl repräsentativer regressionskritischer Testszenarien

Da eine vollständige Automatisierung aller Anwendungsfälle der „Puma“-Webanwendung den Rahmen dieser Studie sprengen würde, wurden repräsentative und regressionskritische Nutzungsszenarien ausgewählt. Diese Szenarien bilden die Kernfunktionalitäten der Anwendung ab und sind nach funktionalen Erweiterungen besonders fehleranfällig.

Für die prototypische Umsetzung wurden folgende drei Szenarien definiert:

1. **Authentifizierung:** Ein Login-Vorgang zur Prüfung des Zugriffs auf geschützte Bereiche. Obwohl dieser Bereich bei funktionalen Erweiterungen der Anwendung selten direkt modifiziert wird, ist er aufgrund seiner hohen Kritikalität (Totalausfall bei Defekt) zwingend abzusichern. Zudem dient er im Framework-Vergleich als methodische Baseline zur Evaluation grundlegender Interaktions- und Wartemechanismen ohne überlagernde Geschäftslogik.

2. **Formularverarbeitung:** Das Ausfüllen, Validieren und Absenden eines typischen Eingabeformulars. Formulare stellen einen zentralen Interaktionspunkt der Anwendung dar. Dieses Szenario eignet sich besonders, um zu vergleichen, wie die Frameworks mit verschiedenen UI-Elementen (wie Dropdowns oder Pflichtfeldern) sowie mit der Auswertung von Validierungsmeldungen im Browser umgehen.
3. **Datensatzbearbeitung:** Ein mehrstufiger Prozess, bei dem ein Datensatz aufgerufen, modifiziert, gespeichert und die erfolgreiche Änderung im Anschluss verifiziert wird. Da die Prozessverwaltung den fachlichen Kernbereich der Anwendung darstellt, sind solche Abläufe nach Updates stark regressionsgefährdet. Dieses komplexe Szenario testet primär, wie zuverlässig die Werkzeuge asynchrone Ladezeiten (z. B. Warten auf das Speichern im Backend) und die zusammenhängende Navigation über mehrere Ansichten hinweg handhaben.

3.4 Ableitung und Gewichtung der Bewertungskriterien

Die Bewertungskriterien für die Nutzwertanalyse wurden direkt aus den fachlichen, technischen und organisatorischen Anforderungen des Ausbildungsbetriebs abgeleitet. Um der betrieblichen Realität gerecht zu werden, wurden folgende Hauptkriterien definiert:

- **Integrationsaufwand:** Bewertung der initialen Einrichtung innerhalb der bestehenden Infrastruktur. Hierbei spielen insbesondere das vorhandene Build-Management (Maven), das genutzte JDK sowie der Umgang mit den strikten Proxy- und Netzwerksperren des Unternehmens eine zentrale Rolle.
- **Verständlichkeit und Wartbarkeit:** Beurteilung der Syntax und Struktur des Testcodes. Es wird bewertet, wie schnell sich neue Entwickler einarbeiten können und wie aufwendig die Pflege der Tests bei zukünftigen UI-Anpassungen ist.
- **Stabilität und Laufzeit:** Analyse der Ausführungsgeschwindigkeit sowie der Zuverlässigkeit der Tests (z. B. Anfälligkeit für sogenannte *Flaky Tests*, die sporadisch ohne Code-Änderung fehlschlagen).
- **Fehlerdiagnose und Dokumentation:** Bewertung der Möglichkeiten zur Fehlersuche (Debugging, Screenshots/Videos bei Fehlschlägen) sowie der Qualität der offiziellen Dokumentation und Community-Unterstützung.

Die Gewichtung dieser Kriterien erfolgt vor dem Hintergrund der Unternehmensziele: Da die Tests von den Entwicklern lokal ausgeführt werden sollen, erhalten Integrationsaufwand und Wartbarkeit eine entsprechend hohe Priorität.

3.5 Vorgehen der prototypischen Umsetzung

Die praktische Erprobung erfolgt schrittweise. Zunächst wird für beide Frameworks eine lokale Testumgebung aufgesetzt. Dabei wird dokumentiert, welche Konfigurationsschritte und Workarounds (beispielsweise bei Abhängigkeiten-Downloads über den Firmen-Proxy) notwendig sind.

Anschließend werden die drei definierten Testszenarien (Login, Formular, Datensatzbearbeitung) nacheinander zunächst in Selenium und danach in Cypress implementiert. Um eine Vergleichbarkeit zu gewährleisten, wird für beide Frameworks derselbe Ausgangszustand der Anwendung genutzt. Während der Entwicklung wird notiert, wie viel Aufwand für das Auffinden von Web-Elementen (DOM-Selektoren), den Umgang mit asynchronen Ladezeiten und den Aufbau der Assertion-Logik (Prüfung der erwarteten Ergebnisse) benötigt wird.

3.6 Datenerhebung und Auswertung

Die Datenerhebung findet parallel zur prototypischen Umsetzung statt. Quantitative Daten wie die Dauer der Testausführung und die Zeilenanzahl des Testcodes (als Indikator für Komplexität) werden direkt gemessen. Qualitative Daten, wie die Qualität der Fehlermeldungen oder Hürden bei der Netzwerkintegration, werden in Form eines Entwicklertagebuchs strukturiert festgehalten.

Nach Abschluss der Implementierung werden die erhobenen Daten in das Kriterienmodell der Nutzwertanalyse überführt. Jedes Framework erhält pro Kriterium eine Bewertung auf einer Skala von 1 bis 10. Diese Punktzahl wird mit der zuvor definierten Gewichtung multipliziert. Die Summe aller gewichteten Punkte ergibt den Gesamtnutzwert, welcher als fundierte Entscheidungsgrundlage für die abschließende Handlungsempfehlung im Fazit dient.

4 Prototypische Umsetzung und Datenerhebung

TODO-Liste für dieses Kapitel:

- **Umgebung:** Java-Version, Cypress- und Selenium-Version sowie den exakten Browser (Microsoft Edge) benennen.
- **Messmethodik:** Die Art der Zeitmessung (z. B. IDE-Logs vs. Cypress Runner) und die Flakiness-Prüfung (Resultate der 10 Durchläufe) konkretisieren.
- **Testdaten & Architektur:** Den Umgang mit Testdaten (Setup/Teardown, Generierung dynamischer Projektnamen) beschreiben und die Architektur-Pattern (Page Object Model in Selenium vs. Custom Commands in Cypress) gegenüberstellen.
- **Code-Beispiele:** Konkrete Snippets für Hilfsmethoden, Custom Commands und Assertions aus beiden Frameworks einfügen.
- **Hürden:** Die Select2/CKEditor-Problematik (Selenium) und die Proxy-Lösung für `npm install` (Cypress) in 1-2 Sätzen prägnant ausformulieren.
- **Visualisierung:** 1-2 aussagekräftige Screenshots ergänzen (z. B. das Cypress Time-Travel Feature oder ein fehlgeschlagener Selenium-Test zur Fehlerdiagnose).

In diesem Kapitel wird die praktische Erprobung der beiden Test-Frameworks beschrieben. Die prototypische Umsetzung erfolgte in der lokalen Entwicklungsumgebung der JUMO GmbH & Co. KG, um die Werkzeuge unter den realen infrastrukturellen Bedingungen des Unternehmens zu evaluieren.

4.1 Das Testobjekt: Die Webanwendung „Puma“

Als Testobjekt dient die intern entwickelte Webanwendung „Puma“. Diese Anwendung wurde geschaffen, um firmeninterne Projekte und Arbeitspläne übersichtlich zu verwalten. Sie integriert komplexe Funktionen wie eine detaillierte Kosten- und Stundenplanung sowie eine SAP-Schnittstelle zur Datensynchronisation. Durch die Zuweisung von Teams und die Einbindung von Dokumenten weist die Benutzeroberfläche eine hohe Dynamik auf. Dies macht sie zu einem idealen, da anspruchsvollen Testobjekt für automatisierte End-to-End-Tests.

4.2 Testkonzeption und Vergleichbarkeit

Um eine valide Datengrundlage für die Nutzwertanalyse zu schaffen, war es zwingend erforderlich, dass die Tests in beiden Frameworks exakt denselben Ablauf abbilden. Beide Implementierungen nutzen denselben Startzustand der Anwendung und führen identische Interaktionsketten durch. Folgende drei repräsentative und regressionskritische Szenarien wurden hierfür umgesetzt:

1. **Authentifizierung (Login):** Überprüfung der Zugriffssteuerung als Basis für alle weiteren Aktionen.

2. **Aktivität anlegen (Einfaches Formular):** Test der grundlegenden Formularverarbeitung und Navigation.
3. **Projekt erstellen (Komplexer Prozess):** Ein mehrstufiger Ablauf mit dynamischen Elementen (z. B. `Select2`-Dropdowns, `CKEditor` für Rich-Text) und asynchronen Speicherprozessen.

4.3 Implementierung mit Selenium

Die Integration von Selenium erfolgte nativ in den bestehenden Java- und Maven-Stack des Unternehmens. Die Einbindung der Abhängigkeiten über die `pom.xml` verlief unkompliziert. Eine technische Hürde stellte die im Betrieb genutzte ältere JDK-Version dar, welche die Auswahl einer dazu kompatiblen Selenium-Version für das `ui-test`-Modul erforderte.

Die Teststruktur wurde mit dem Test-Runner JUnit abgebildet. Bei der Implementierung zeigte sich, dass Selenium stark technisch orientiert ist. Web-Elemente mussten explizit über Selektoren gesucht und als `WebElement` instanziiert werden, bevor Aktionen darauf ausgeführt werden konnten. Besonders bei dynamischen UI-Komponenten wie `Select2`-Feldern oder dem Rich-Text-Editor wurde der Code schnell unübersichtlich. Um die Wartbarkeit zu erhalten, war es zwingend notwendig, wiederkehrende Abläufe in eigene Hilfsmethoden (z. B. `selectDropdownByTextAndAssert()` oder `select2SearchAndAssert()`) auszulagern. Zudem musste intensiv mit expliziten Wartebedingungen (`wait.until()`) gearbeitet werden, um Timing-Probleme bei nachladenden Seitenelementen zu verhindern.

4.4 Implementierung mit Cypress

Im Gegensatz zu Selenium basiert Cypress auf Node.js und npm. Dies führte im stark reglementierten Firmennetzwerk der JUMO GmbH & Co. KG direkt zu Beginn zu Problemen, da der Befehl `npm install` durch Proxy- und Sicherheitsrichtlinien blockiert wurde. Die Installation erforderte daher zunächst die Einbindung des IT-Services, was den initialen organisatorischen Integrationsaufwand erhöhte.

Nach der Einrichtung überzeugte Cypress durch eine sehr direkte und deskriptive Syntax. Die Tests ließen sich stark an der tatsächlichen manuellen Nutzerinteraktion orientieren (z. B. Element suchen, tippen, prüfen über Befehlsketten). Dynamische Ladezeiten wurden durch das integrierte Auto-Waiting und das Abfangen von Ajax-Requests via `cy.intercept()` und `cy.wait()` robust gehandhabt, ohne dass statische Pausen im Code ergänzt werden mussten. Auch hier wurden zur Steigerung der Lesbarkeit wiederkehrende Schritte, wie der Login-Prozess, in Custom Commands (`cy.login()`) ausgelagert.

4.5 Datenerhebung und praktische Beobachtungen

Während der Testausführung wurden parallel quantitative und qualitative Daten für die Auswertung erhoben:

- **Laufzeit:** Beide Tools benötigten für den initialen Startlauf (inklusive Start der Testumgebung und Aufbau des Startzustands) rund 45 Sekunden. Bei einfachen Folgeausführungen (Aktivität anlegen) lagen beide Frameworks mit 22 bis 32 Sekunden eng beieinander. Beim vollständigen Erstellen eines Projekts wirkte Cypress durch die internen Retry-Mechanismen etwas performanter.
- **Stabilität (Flakiness):** Bei zehn aufeinanderfolgenden Testläufen wies Selenium eine Fehlerquote von rund 40 % (4 von 10) auf. Die Abbrüche waren meist auf minimale Timing-Abweichungen oder nicht vollständig geladene Elemente zurückzuführen. Cypress hingegen lief deutlich stabiler mit lediglich einem Fehlschlag bei zehn Durchläufen.
- **Fehlerdiagnose:** Tritt in Selenium ein Fehler auf, wird meist eine technische Exception (z. B. in Eclipse) ausgegeben. Cypress bot hier durch den grafischen Test Runner und die Time-Travel-Funktion, mit der Zustände der Anwendung zum Zeitpunkt des Fehlers visuell rekonstruiert werden können, einen signifikanten Mehrwert bei der Fehleranalyse.

5 Ergebnisse und Nutzwertberechnung

5.1 Präsentation der quantitativen Messdaten

5.2 Mathematische Auswertung (Nutzwertmatrix)

5.3 Analyse und Diskussion der Ergebnisse

6 Fazit und Handlungsempfehlung

6.1 Zusammenfassung der Erkenntnisse

6.2 Handlungsempfehlung für die JUMO GmbH & Co. KG

6.3 Ausblick und zukünftige Schritte

A Anhang

A.1 Anhang A

Das hier ist ein Anhangs-Kapitel. Es kann wie die anderen auch strukturiert und erweitert werden.

Literaturverzeichnis

- [1] A. Spillner und T. Linz, *Basiswissen Softwaretest: Aus-und Weiterbildung zum Certified Tester–Foundation Level nach ISTQB-Standard*. dpunkt. verlag, 2024.
- [2] Selenium Project, *Selenium Documentation*, <https://www.selenium.dev/documentation/>, [Online; abgerufen am 13.05.2026], 2024.
- [3] Cypress.io, *Cypress Documentation*, <https://docs.cypress.io/>, [Online; abgerufen am 13.05.2026], 2024.
- [4] P. Liggesmeyer, *Software-Qualität: Testen, Analysieren und Verifizieren von Software*, 2. Spektrum Akademischer Verlag, 2009.
- [5] J. B. Kühnapfel, *Scoring und Nutzwertanalysen: Ein Leitfaden für die Praxis*. Springer, 2021.